

Rahnuma

रहनुमा رہنما

A working guide to the ILM, PANINI and Zistgah stack, from bytes and scripts to compilers, Pāṇini and quantum substrates

प्रकृति

	written	Devanagari hub	Romenagri	and back
Devanagari	प्रकृति	→ प्रकृति	→ prak_ri_ti	→ प्रकृति
Telugu	ప్రకృతి	→ प्रकृति	→ prak_ri_ti	→ ప్రకృతి
Bengali	প্রকৃতি	→ प्रकृति	→ prak_ri_ti	→ the same identity
Urdu, as usually written	ہندوی	→ हनदवी	→ _hana_dawee	→ हनदवी
Urdu, with zer and sukun	ہندویٰ	→ हिन्दवी	→ _hin_dawee	→ हिन्दवी

One identity, many scripts. Telugu and Bengali meet Devanagari in one hub, become the same Romenagri word, and come back exactly. Urdu meets the hub at what is written: without its short-vowel marks the word reads hanadawee; with zer and sukun it is hindawi. Every cell here was produced by the Hindawi tools while this guide was built.

Abhishek Choudhary, AyeAI. Version 2.0.0, 3 October 2026.

Rahnuma, the edition for: College student

Contents

Part I: Orientation 2

1. How to use this guide	3
3. Start here: find your way in	4

Part III: Setting up 6

13. Choosing a platform	7
14. The baseline environment and a first run	10
15. Running every component	14

Part V: Working with the estate 26

26. GATE: sit it, whatever your age	27
27. Syllabus links, and our syllabus	29

Part I: Orientation

1. How to use this guide

In plain words. This guide explains a large body of work on computers, languages and intelligence. You can read it, run the programs on your own computer, or ask an AI to help you, and every example in it was really run.

This guide is for anyone who wants to use, check or extend the work gathered in the *zistgah*, *project-ilm* and *hindawiai* organisations on GitHub: students preparing for GATE, engineers who write compilers, firmware or chip layouts, linguists who work with scripts and sound, and researchers in quantum computing and AI. It asks for curiosity and a laptop, and nothing else.

There are four ways in.

- **Understand.** Part II gives the whole architecture first: where the work comes from, how the parts compose, each term with its nearest analogue and the difference, and the three Acts.
- **Read.** Parts IV and V explain the ideas the work rests on, with the mathematics written out: how numbers and letters sit in memory, what makes a transliteration reversible, how Pāṇini's grammar computes, how a compiler turns text into machine instructions, and how an oscillator network and a quantum circuit attack the same problem.
- **Run.** Part III gives the exact commands to set up Linux, Windows with WSL, or macOS, and to run every component of the estate on your own machine.
- **Ask.** Chapter 24 shows how to put any AI assistant to work on these repositories, and how to check what it tells you.

Conventions used throughout:

- Every example that could run while this guide was being built did run, and the text shows its real output, marked *Executed while this guide was built*. The package's own `gate`, `bash ops/verify.sh`, runs those examples again and fails if any output differs from what is printed here.
- Status words are used exactly. **Tested:** a test in the repository asserts it. **Executed:** it ran and its output is shown. **Written:** the code exists but has not been run here. **Mocked:** a stand-in replaces hardware that is not attached. **Planned:** described, not built.
- Everything said about the repositories was retrieved from the repositories themselves on 29 September 2026: their listings on GitHub, their `CITATION.cff` files and their READMEs. Where a repository does not record something, this guide says so instead of filling the gap.
- A dagger, †, marks a statement that follows the author's working notes and write-ups as supplied on 29 September and 3 October 2026, rather than the repositories.
- Sanskrit is transliterated in IAST. The spelling is British.

The work lives at github.com/zistgah, github.com/project-ilm and github.com/hindawiai, on the site zistgah.org, and in the Zenodo records listed in Chapter 12.

3. Start here: find your way in

In plain words. Four steps to find your part of the guide: who you are, your board or field, what you want to do, and your path with a PDF made for you.

This guide serves a school student and a senior researcher; a village health worker and an investor. Nobody needs all of it. Four steps find your part, on the site or with this page:

1. **Who you are.** A school student, a college student, a postgraduate or doctoral researcher; a teacher; an academic researcher; a working professional, someone in public service, policy or law, someone in community, NGO or village work, an artist, writer or maker; an industry partner or investor; or an AI agent working for a person.
2. **Your board or your field.** A school student chooses a board, ICSE, ISC, CBSE, IGCSE, IB, a state board or another; and a class. Everyone else chooses a field from the international standard classification of fields of study, ISCED-F 2013, the same one FAKIR's lattice uses. It places each subject where its own scholars would: political science is under social and behavioural sciences, not the humanities; law is under business, administration and law; linguistics is under languages.
3. **What you want to do.** Learn the basics, run the software, study or research, teach, contribute, or adopt, fund or invest.
4. **Your path and your edition.** The site lists your chapters in order, with reading times and the components to try first, and offers a PDF edition made for you. Every edition below is a printed selection from this one guide, so all of them stay in step.

Edition	PDF
School student	role-school.pdf
College student	role-college.pdf
Postgraduate or doctoral researcher	role-pg.pdf
Teacher or lecturer	role-teacher.pdf
Academic researcher	role-researcher.pdf
Working professional or engineer	role-professional.pdf
Public service, policy or law	role-public.pdf
Community, NGO or village work	role-community.pdf
Artist, writer or maker	role-creator.pdf
Industry partner, investor or funder	role-industry.pdf
Generic programmes and qualifications	field-00.pdf
Education	field-01.pdf
Arts and humanities	field-02.pdf
Social sciences, journalism and information	field-03.pdf

Edition	PDF
Business, administration and law	field-04.pdf
Natural sciences, mathematics and statistics	field-05.pdf
Information and Communication Technologies (ICTs)	field-06.pdf
Engineering, manufacturing and construction	field-07.pdf
Agriculture, forestry, fisheries and veterinary	field-08.pdf
Health and welfare	field-09.pdf
Services	field-10.pdf

The first public version of this guide, written for advanced researchers and senior professionals, stays as it was at [version 1](#).

Part III: Setting up

13. Choosing a platform

In plain words. Which computer set-up to use: Linux works best, Windows works through WSL, and a Mac works for most things.

Platform	What works	What is limited
Linux, native: Ubuntu 24.04 LTS or 26.04 LTS	Everything in the estate: shell scripts, compilers, Docker, GPU work, serial devices	Nothing; this is the reference platform
Windows 10 or 11 with WSL 2	Nearly everything, inside a real Ubuntu	USB devices need usbipd-win; the GPU is reached through the Windows driver; files under <code>/mnt/c</code> are slow
Windows, native	Reading, the browser-based tools, editing, Python, Node	The estate's bash scripts, Docker Engine, OpenTimestamps and systemd services do not run natively
macOS	Most command-line work, through Homebrew	No CUDA; BSD versions of the core tools unless GNU versions are installed

The estate's own workstation is a Ryzen 7 laptop with 16 GB of memory and an RTX 3050 with 4 GB, running Ubuntu with Docker; and TransEg names the same target. Any machine with 8 GB of memory and 30 GB of free disk runs every example in this guide.

Installing Ubuntu on a PC

1. Download the desktop image and the `SHA256SUMS` file from ubuntu.com/download/desktop into one folder.
2. Check the download. The command must print `OK` against the image's name:

```
sha256sum -c SHA256SUMS --ignore-missing
```

3. Write the image to a USB stick of 8 GB or more. On Windows, use Rufus or balenaEtcher. On Linux, find the stick first; the commands below assume it is `sdb`, and they erase it:

```
lsblk -d -o NAME,SIZE,MODEL
ISO=$(ls ubuntu-*-desktop-amd64.iso)
sudo dd if="$ISO" of=/dev/sdb bs=4M status=progress oflag=sync
```

4. Boot from the stick (the boot-menu key is usually F12, F10, F2 or Esc), choose *Install Ubuntu*, and either erase the disk or install alongside Windows. Before installing alongside Windows, turn off Windows Fast Startup, and if BitLocker is on, save its recovery key or suspend it. Secure Boot can stay on; Ubuntu's boot loader is signed.
5. After the first login:

```
sudo apt update && sudo apt full-upgrade -y
```

Windows with WSL 2

In a PowerShell window run as Administrator:

```
wsl --install
```

Restart, then:

```
wsl --list --online  
wsl --install -d Ubuntu-24.04  
wsl --set-default-version 2  
wsl --list --verbose
```

The last command must show `VERSION 2` beside Ubuntu. Open Ubuntu from the Start menu, create your Linux user and update it with `sudo apt update && sudo apt full-upgrade -y`. Then switch on systemd, which Docker Engine and several estate services expect:

```
printf '[boot]\nsystemd=true\n' | sudo tee /etc/wsl.conf
```

Run `wsl --shutdown` in PowerShell and open Ubuntu again; `systemctl is-system-running` now answers `running` or `degraded`.

Five rules that save hours:

- Keep your work in the Linux file system, under `~/work`, never under `/mnt/c`; file access across that boundary is many times slower. Windows reaches your Linux files at `\\wsl$`.
- Use VS Code on Windows with its WSL extension, and type `code .` inside a Linux folder.
- For an NVIDIA GPU install only the Windows driver; do not install a Linux display driver inside WSL. `nvidia-smi` inside Ubuntu then lists the card.
- To hand a USB device such as an ESP32 board to WSL, install `usbipd-win` with `winget install usbipd`, then in an Administrator PowerShell run `usbipd list`, `usbipd bind --busid 1-4` and `usbipd attach --wsl --busid 1-4`, using the bus identifier that `usbipd list` printed for your device in place of `1-4`.
- To cap WSL's memory, create a file named `.wslconfig` in your Windows user folder containing the two lines `[wsl2]` and `memory=8GB`.

Windows, native

```
winget install --id Git.Git -e  
winget install --id Python.Python.3.12 -e  
winget install --id OpenJS.NodeJS.LTS -e
```

```
winget install --id Microsoft.VisualStudioCode -e
git config --global core.autocrlf input
```

The estate's scripts are bash scripts written for GNU tools. Git Bash runs simple ones, but not systemd services, Docker Engine or OpenTimestamps workflows. flex and bison come through MSYS2, with `pacman -S --needed base-devel flex bison mingw-w64-ucrt-x86_64-gcc`. Windows file systems ignore case, lock open files and end lines with CRLF, and each of these breaks scripts written for Linux; the `autocrlf input` setting above prevents the third. Native Windows is right for reading, for the browser-based tools (the PANINI studio, CHAKRA, Jyotish, the Zamin simulator) and for Python notebooks. Build and run everything else in WSL.

macOS

```
xcode-select --install
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install git python@3.12 node flex bison gcc make
echo 'export PATH="$(brew --prefix bison)/bin:$(brew --prefix flex)/bin:$PATH"' >> ~/.zshrc
```

macOS ships a bison from 2006 and BSD versions of `sed`, `grep` and `date`. Homebrew installs flex and bison “keg-only”, that is, not on the PATH, and the last line puts them first. There is no CUDA on current Macs; Docker runs through Docker Desktop or Colima.

14. The baseline environment and a first run

In plain words. How to install the basic tools and run your first real program from this work.

Packages

On Ubuntu, native or in WSL:

```
sudo apt update
sudo apt install -y build-essential git curl wget unzip make cmake pkg-config \
    flex libfl-dev bison gawk gdb python3 python3-venv python3-pip pipx nodejs npm jq \
    fonts-noto-core fonts-indic
gcc --version | head -1; flex --version; bison --version | head -1
python3 --version; node --version
```

This brings GCC and binutils, the lexer and parser generators of Chapter 21 with flex's library and the gawk that Hindawi's driver needs, Python with virtual environments, Node for the JavaScript engines of PANINI, CHAKRA and Jyotish, and fonts for the Indic scripts.

Git and GitHub

Use the name and email of your own GitHub account in place of the example ones:

```
git config --global user.name "Ada Lovelace"
git config --global user.email "ada@example.org"
git config --global init.defaultBranch main
ssh-keygen -t ed25519 -C "ada@example.org"
cat ~/.ssh/id_ed25519.pub
```

Paste the printed key at GitHub under *Settings, SSH and GPG keys*, and check it with `ssh -T git@github.com`. The GitHub CLI turns forks and pull requests into single commands:

```
sudo apt install -y gh
gh auth login
```

Docker Engine

These are Docker's own commands for Ubuntu, from docs.docker.com/engine/install/ubuntu:

```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker "$USER"
```

Log out and back in, then `docker run hello-world` must greet you.

A first run: PANINIq

PANINIq, the quantum-simulation backend of PANINI, is a good first repository: small, tested, and with its own gate. Its README gives the install; the lines below follow it, with the virtual environment kept outside the clone:

```
mkdir -p ~/work && cd ~/work
git clone https://github.com/zistgah/paniniq.git
python3 -m venv ~/work/venv-paniniq
. ~/work/venv-paniniq/bin/activate
cd paniniq
export PYTHONDONTWRITEBYTECODE=1
pip install -r requirements.txt qiskit
python3 -m unittest discover -s tests
python3 demo.py
bash ops/verify.sh
```

The environment variable and the outside virtual environment matter because the repository's gate counts any `__pycache__` folder or compiled `.pyc` file inside the clone as a build artefact (its clause P08). Python writes those caches when it runs the tests, and a virtual environment inside the clone holds thousands of them, so either would turn the gate red for a reason that has nothing to do with the code.

This is the run made while this guide was built, on the repository at commit 82a5cf2:

Recorded run: `data/logs/paniniq-run-2026-09-29.txt`

```
$ git log -1 --format='%h %ad' --date=short
82a5cf2 2026-09-28

$ export PYTHONDONTWRITEBYTECODE=1

$ python3 -c 'import numpy, scipy, qiskit; print(numpy.__version__, scipy.__version__,
qiskit.__version__)'
2.5.3 1.18.1 2.5.2

$ python3 -m unittest discover -s tests
-----
Ran 23 tests in 0.217s

OK

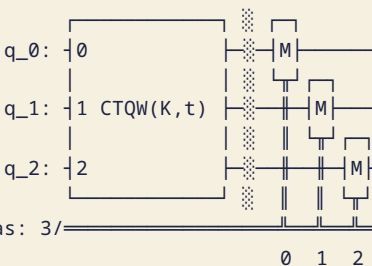
$ bash ops/verify.sh
Panini Q: contract verification
PASS    P01  every source carries the copyright and SPDX lines
PASS    P02  no affiliation other than AyeAI is claimed
PASS    P03  the test suite passes
PASS    P04  the suite catches the anti-ferromagnetic zeroing bug
PASS    P05  demo.py runs to completion
PASS    P06  LICENSE is the FSF's verbatim GPL-3.0 text
```

PASS	P07	every row the README marks Tested names its test
PASS	P08	no build artefacts
PASS	P09	no path outside the folder is named
PASS	P10	misty.json and CITATION.cff agree; no placeholder DOI
10 passed, 0 failed, 0 unjudged		

And the demonstration, as it printed:

Recorded run: data/logs/paniniq-demo-2026-09-29.txt

```

$ python3 demo.py
=====
1. Point events -> inclination field -> relaxation -> spawn
=====
event 0: point=[0.85, 0.12, 0.44]
    order parameter R = 0.3874    |S| = 5    -> SPAWNED a node
event 1: point=[0.1, 0.9, 0.3]
    order parameter R = 0.5859    |S| = 5
event 2: point=[0.83, 0.14, 0.46]
    order parameter R = 0.4573    |S| = 6    -> SPAWNED a node
=====
2. Lifecycle maintenance (decay priors, then prune/consolidate)
=====
|S| before maintenance: 8
pruned=1, merged=0
|S| after maintenance: 7
=====
3. Compile to Qiskit + statevector validation
=====


```

graph LR
 q0[q_0: 0] --- CTQW[CTQW(K, t)]
 q1[q_1: 1] --- CTQW
 q2[q_2: 2] --- CTQW
 CTQW --- M0[M]
 CTQW --- M1[M]
 CTQW --- M2[M]
 M0 --- meas0[0]
 M1 --- meas1[1]
 M2 --- meas2[2]
 meas0 --- meas[meas: 3/]
 meas1 --- meas
 meas2 --- meas

```



statevector dim = 8, sums to 1.000000 (unitarity check)


=====
4. Max-Cut benchmark: oscillator vs. brute force vs. QAOA
=====


| Method     | cut_value | time     |
|------------|-----------|----------|
| oscillator | 4.0       | 2.90 ms  |
| bruteforce | 4.0       | 0.22 ms  |
| qaoa       | 4.0       | 52.09 ms |


=====
5. Hardware bridge (mocked -- no real serial device exists here)
=====
21 frame(s) 'sent' to MOCK
sample frame (hex): aa55000100b3
This only proves the K-matrix -> byte-frame packing is correct.
there's no serial port or USB device available in this sandbox.
=====
6. ClassicalSubstrate (PyTorch) scaling demo
=====
SKIPPED: torch is not installed in this sandbox (no network access)

```

```
pedler/substrate.py::ClassicalSubstrate is written and ready -- `pip install torch` and re-run this script; it will automatically use a GPU if `torch.cuda.is_available()`.
```

Chapter 23 explains every number in it: the order parameter R , the spawned states, the continuous-time quantum walk on three qubits, and why the oscillator, brute force and QAOA agree on a cut of 4.

Every other component

Chapter 15 gives the commands for every component of the estate: the ones executed for this guide, and the rest as their own READMEs state them.

15. Running every component

In plain words. Step-by-step commands to start each part on your own computer, with what you should see when it works and what to do when it does not.

Every component below was cloned and run on a clean machine on 3 October 2026, with the commands its own README gives, and each card says exactly what happened. Where a step needed something the README does not mention, the card adds it; where a component did not pass, the card says why. Set up the machine first (Chapters 13 and 14).

Each card has the same shape: what the component is, what it needs, the commands to type in order, what you should see, and its status. Every card starts with a clone of the repository into a folder of the same name; the commands that follow are typed inside that folder.

Machine: Ubuntu 24.04, Python 3.12, Node 22, GCC 13. *Run on:* 3 October 2026.

Language and identity

Hindawi and Romenagri

What it is. The Hindawi Programming System's C front end, its driver hincc and the Romenagri script layer. *Needs:* gcc, make, flex, libfl-dev, gawk.

```
git clone https://github.com/hindawiai/chintamani.git && cd chintamani
mkdir -p .root/usr/bin .root/usr/lib .root/usr/include
make -C Romenagri all && make -C Romenagri install INSTRROOT="$PWD/.root"
export PATH="$PWD/.root/usr/bin:$PATH"
make -C Hindawi/guru all && make -C Hindawi/guru install INSTRROOT="$PWD/.root"
make -C Hindawi/hindr install INSTRROOT="$PWD/.root"
cp Hindawi/samples/HindiC.uhin . && hincc HindiC.uhin
printf 'रम\nरम\n' | ./hin.exe
```

You should see: संकलन के परिणाम, then the program asks आपका नाम क्या है? and counts to ten. *Status:* Executed for this guide. The compilers chapter prints the run.

The Urdu edition

What it is. Urdu shailis and the driver urducc, which sends an Urdu-script program through the Devanagari hub and Romenagri. *Needs:* the Hindawi build above, on the PATH.

```
git clone https://github.com/zistgah/urdu-ilm.git && cd urdu-ilm
mkdir -p .root/usr/bin .root/usr/lib .root/usr/include
make -C Romenagri all && make -C Romenagri install INSTRROOT="$PWD/.root"
export PATH="$PWD/.root/usr/bin:$PATH:<chintamani clone>/.root/usr/bin"
make -C ILM/guru all
bash ILM/urducc -s ILM/samples/UrduC.uhin
bash ILM/urducc ILM/samples/UrduC.uhin && ./hin.exe
```

You should see: C with the names na and ka, then the prompt in the hub script. *Status:* Executed for this guide. Replace with the folder of the Hindawi clone; the compilers chapter prints the run.

tajziya

What it is. The parser federation for the classical languages, by family. *Needs:* python3, make.

```
git clone https://github.com/zistgah/tajziya.git && cd tajziya
PYTHONPATH=src python3 -m tajziya commonalities
make check
```

You should see: make check: 19 passed, 0 failed, 1 unjudged. *Status:* Passed when run on 3 October 2026. Run the module with PYTHONPATH=src, from the clone's root.

The PANINI family

PANINI, the prompt-cycle language

What it is. The language the cyclers are written in, with its engine and studio. *Needs:* python3, node.

```
git clone https://github.com/zistgah/panini.git && cd panini
python3 panini.py check cyclers/pench.pni
python3 panini.py stages cyclers/yadein.pni
node tests/panini.test.mjs
python3 panini.py serve
```

You should see: 0 problems; 45 pass, 0 fail; the studio answers on 127.0.0.1:8717. *Status:* Passed when run on 3 October 2026.

PANINI, parser and interpreter

What it is. A general-purpose PANINI implementation with its conformance report. *Needs:* node.

```
git clone https://github.com/zistgah/panini_by_claude.git && cd panini_by_claude
node bin/panini.mjs conformance
npm test
```

You should see: 145 passed, 0 failed. *Status:* Passed when run on 3 October 2026.

PANINI, the stage-0 bootstrap

What it is. PANINI with a bootstrap that records its own evidence. *Needs:* node.

```
git clone https://github.com/zistgah/panini_by_grok.git && cd panini_by_grok
node src/cli.js run examples/hello.pni
node tests/run.mjs
node scripts/bootstrap.mjs
```

You should see: Hello, PANINI; 32 passed, 0 failed; stage-0 evidence written. *Status:* Passed when run on 3 October 2026. Its wider report, npm test, passes 81 of 83 here; the two others need tools this machine lacks.

The Humanesque merged release

What it is. PANINI, ILM, Hindawi, the cyclers and Mez in one tree. *Needs:* bash, node; optionally zig.

```
git clone https://github.com/zistgah/humanesque.git && cd humanesque
bash VERIFY.sh
npm test
```

You should see: 79 of 83 checks pass. *Status:* Partly passes, as run on 3 October 2026. The four that do not pass here need two downloadable model files (wllama.wasm and stories15M.Q4_0.gguf), the zig toolchain, and the Bengali flattening round trip.

PANINIq

What it is. The oscillator substrate, the PEDLER engine and quantum validation. *Needs:* python3, numpy, scipy, qiskit.

```
git clone https://github.com/zistgah/paniniq.git && cd paniniq
python3 -m venv ../venv-paniniq && . ../venv-paniniq/bin/activate
export PYTHONDONTWRITEBYTECODE=1
pip install -r requirements.txt qiskit
python3 -m unittest discover -s tests
python3 demo.py
bash ops/verify.sh
```

You should see: Ran 23 tests, OK; its gate 10 passed. *Status:* Executed for this guide. The first-run chapter prints the run.

PANINIB, biological realization

What it is. Biological intent lowered through typed IRs to a realization plan. *Needs:* python3.

```
git clone https://github.com/zistgah/paninib.git && cd paninib
python3 panini compile examples/replace.panini -o out
python3 panini inspect out --stage sequence-ir
python3 panini verify out
```

You should see: each step prints its JSON record. *Status:* Passed when run on 3 October 2026.

PANINIphy, physical realization

What it is. Physical intent compiled through a physical IR, with parts chosen by requirement. *Needs:* python3.

```
git clone https://github.com/zistgah/paniniphy.git && cd paniniphy
python3 panini compile examples/modular_block.panini -o out
python3 panini parts --category actuator --force 50N --stroke 20mm --voltage 12V
```

You should see: physical resolution=RESOLVED verify=True. *Status:* Passed when run on 3 October 2026.

Cyclers, desk and studio

matba, the press

What it is. Posters in, book out, sealed, pushed, minted. *Needs:* python3.

```
git clone https://github.com/zistgah/matba.git && cd matba
python3 matba.py serve
```

You should see: the press answers on 127.0.0.1:8710. *Status:* Passed when run on 3 October 2026.

khwab, the cutting room

What it is. The visual cycler. *Needs:* python3.

```
git clone https://github.com/zistgah/khwab.git && cd khwab
python3 khwab.py serve
```

You should see: the cutting room answers on 127.0.0.1:8711. *Status:* Passed when run on 3 October 2026.

awaz, the listening room

What it is. The audio cycler. *Needs:* python3.

```
git clone https://github.com/zistgah/awaz.git && cd awaz
python3 awaz.py serve
```

You should see: the listening room answers on 127.0.0.1:8712. *Status:* Passed when run on 3 October 2026.

tilasm, pench and yadein

What it is. The immersive, embodied and record cyclers; each repository has the same tests and its own page. *Needs:* node.

```
git clone https://github.com/zistgah/tilasm.git && cd tilasm
node tests/cycler.test.mjs
node tests/inbox.test.mjs
node tests/models.test.mjs
```

You should see: 20, 23 and 22 pass, 0 fail. *Status:* Partly passes, as run on 3 October 2026. Run the same three in zistgah/pench and zistgah/yadein. Two further test files, genie.test.mjs and leak.test.mjs, import modules the repositories do not carry.

GENIE

What it is. The Generalized Emotive-Narrative Interaction Engine. *Needs:* node.

```
git clone https://github.com/zistgah/genie.git && cd genie
node test/run.js
```

You should see: 191 passed, 0 failed. *Status:* Passed when run on 3 October 2026. Open index.html, or its page, for the engine itself.

alam

What it is. Small work, cycled with any AI. *Needs:* node, npm.

```
git clone https://github.com/zistgah/alam.git && cd alam
npm install jsdom
node test/alam-test.js
node test/dom-test.js
```

You should see: alam dom: 9 passed, 0 failed. *Status:* Passed when run on 3 October 2026. The page test needs jsdom, installed by the first line.

Mez, the Cognitive Workbench

What it is. The local-first desk. *Needs:* python3.

```
git clone https://github.com/zistgah/mez.git && cd mez
./mez doctor
./mez bearings
./mez serve
```

You should see: no account, no API key, no network required; the desk answers on 127.0.0.1:7373. *Status:* Passed when run on 3 October 2026.

Knowledge, domains and instruments

FAKIR

What it is. The knowledge kernel and its explorer. *Needs:* bash.

```
git clone https://github.com/zistgah/fakir.git && cd fakir
bash ops/verify.sh
```

You should see: CONTRACT SATISFIED, 6 checks. *Status:* Passed when run on 3 October 2026. The explorer is its page: zistgah.org/fakir

Dhancha

What it is. The domain spine every FAKIR domain is built on. *Needs:* bash, python3, make.

```
git clone https://github.com/zistgah/dhancha.git && cd dhancha
make check
```

You should see: site matches the registries; pr_gate bites on a claim and passes a clean change. *Status:* Passed when run on 3 October 2026.

ertabat

What it is. The communication-engineering domain. *Needs:* bash.

```
git clone https://github.com/zistgah/ertabat.git && cd ertabat
bash ops/verify.sh
```

You should see: CONTRACT OK. *Status:* Passed when run on 3 October 2026.

jugaad28

What it is. The 28 nm tape-out catalogue. *Needs:* bash.

```
git clone https://github.com/zistgah/jugaad28.git && cd jugaad28
bash ops/verify.sh
```

You should see: VERIFY GREEN. *Status:* Passed when run on 3 October 2026.

TransEg

What it is. The local digital-twin reference implementation. *Needs:* bash, python3; Docker for the stand-up.

```
git clone https://github.com/zistgah/transeg.git && cd transeg
./scripts/test.sh
./scripts/bootstrap.sh --dry-run
```

You should see: 10 passed; the stand-up plan. *Status:* Passed when run on 3 October 2026. The full stand-up, ./scripts/bootstrap.sh and docker compose, needs Docker and a GPU; it was not run here.

TransEg identity governance

What it is. Typed layers, envelope, staged policies and audit. *Needs:* python3, jsonschema, pytest.

```
git clone https://github.com/zistgah/transeg-idgov.git && cd transeg-idgov
pip install jsonschema pytest
python3 tools/idgov_validate.py policies
python3 -m pytest -q tests/
```

You should see: VALID: 8 stage policies; 14 passed. *Status:* Passed when run on 3 October 2026.

CHAKRA

What it is. The offline observatory and its calendars. *Needs:* node.

```
git clone https://github.com/project-ilm/chakra.git && cd chakra
node test/run.js
```

You should see: ALL SUITES PASSED. *Status:* Passed when run on 3 October 2026. Open index.html for the observatory.

Jyotish

What it is. The panchang, the Kaivalyik projection of CHAKRA. *Needs:* bash, node.

```
git clone https://github.com/zistgah/jyotish.git && cd jyotish
bash ops/verify.sh
```

You should see: PASSED, with 1 check skipped and named. *Status:* Passed when run on 3 October 2026. Open index.html for the panchang.

Cognition and hardware

PRATIK kernel

What it is. The balanced-ternary, event-driven kernel. *Needs:* g++; nvcc for CUDA.

```
git clone https://github.com/zistgah/pratik_core_mvp.git && cd pratik_core_mvp
bash build.sh
bash build.sh cuda
```

You should see: ALL CHECKS PASSED. *Status:* Passed when run on 3 October 2026. The CPU build passed here; the CUDA build needs nvcc and an NVIDIA GPU. The script is not marked executable, so call it with bash.

Zamin

What it is. Ternary silicon: SPICE cells, layout and a simulator. *Needs:* ngspice, python3.

```
git clone https://github.com/zistgah/zamin.git && cd zamin
cd spice && ngspice -b sign_frustration_tb.sp
```

You should see: a SPICE run of the ternary cells. *Status:* Does not run as is, as run on 3 October 2026. On ngspice 42 the testbench stops: the source v_gnd is reported as a shorted voltage source. The interactive simulator is its page.

QEDLER

What it is. The event-hypergraph physics programme. *Needs:* bash.

```
git clone https://github.com/project-ilm/qedler.git && cd qedler
bash ops/verify.sh
bash ops/resume.sh && cat RESUME.md
```

You should see: 5 of 6 clauses pass. *Status:* Partly passes, as run on 3 October 2026. Clause C2, manifest completeness, fails: 15 files are not in MANIFEST.sha256. Resealing the manifest clears it.

Provenance, tools and labs

Misty DOI

What it is. DOI minting and packaging for Zenodo. *Needs:* python3, pip.

```
git clone https://github.com/project-ilm/misty-doi.git && cd misty-doi
pip install misty-doi
misty --help
misty init
misty validate -m misty.json
```

You should see: `usage: misty [-h] [--version] {init,validate,transform,package,ots,publish}`. *Status:* Passed when run on 3 October 2026. Publishing needs a Zenodo token in ZENODO_TOKEN.

Research Kundali

What it is. A research map of a person, a lab or a paper. *Needs:* python3 and network access to the public records it reads.

```
git clone https://github.com/project-ilm/research-kundali.git && cd research-kundali
python3 kundali/kundali.py <ORCID, lab name or DOI> --out out
```

You should see: a research map in out/. *Status:* Not run here: needs network access. It reads public records such as ORCID; it could not reach them from this build machine, and stops rather than write an empty chart.

The linguistics lab

What it is. A starter lab in a container. *Needs:* Docker.

```
git clone https://github.com/project-ilm/linguistics-labs.git && cd linguistics-labs
docker build -t ilm-lab .
docker run --rm -p 8888:8888 -v "$PWD":/lab ilm-lab
```

You should see: the lab on localhost:8888. *Status:* Not run here: needs Docker. Not run here: this build machine has no Docker.

This guide

What it is. The site, the PDF and the Markdown, with their gate. *Needs:* python3, gcc, make, flex, libfl-dev, gawk, gdb; for a rebuild also pandoc, node, graphviz.

```
git clone https://github.com/zistgah/rahnuma.git && cd rahnuma
bash ops/verify.sh
bash examples/hindawi.sh words
python3 tools/build.py
```

You should see: 12 passed, 0 failed, 0 unjudged. *Status:* Passed when run on 3 October 2026.

Components you open in a browser

Component	Page
FAKIR's explorer	zistgah.org/fakir
The shared dome	zistgah.github.io/dome
AAB, the factory process	zistgah.org/aab
GENIE	zistgah.github.io/genie

tilasm, pench and yadein | zistgah.github.io/tilasm |
Kitab | zistgah.github.io/kitab |
Tok DOI | project-ilm.github.io/tok-doi |
CHAKRA | project-ilm.github.io/chakra |
Jyotish | zistgah.github.io/jyotish |
ILM | ilm.codes |
PANINI's studio | zistgah.github.io/panini |
The estate | zistgah.org |

Part V: Working with the estate

26. GATE: sit it, whatever your age

In plain words. India's national engineering examination: what it covers, how to apply, and why it is worth sitting at any age.

The facts for 2027

The Graduate Aptitude Test in Engineering 2027 is organised by IIT Madras. There are 30 test papers, including Robotics and Automation (RA) for the first time; a candidate may sit one paper or an allowed pair of two. The regular registration, first due to close on 27 September 2026, has been extended to 5 October 2026 without a late fee, and the extended registration, with a late fee, closes on 12 October 2026, all on the portal gate2027.iitm.ac.in. Registration goes through DigiLocker for Indian nationals. The application correction window runs from 14 to 21 October 2026, the examination is held in February 2027, and results are due on 19 March 2027. There is no upper age limit: a candidate needs to be in the third or a later year of an undergraduate degree, or to have completed one. A GATE score is valid for three years and opens postgraduate admissions and recruitment in public-sector undertakings.

The official syllabi

- Computer Science and Information Technology (CS): [CS_GATE2027_Syllabus.pdf](#)
- Robotics and Automation (RA): [RA_GATE2027_Syllabus.pdf](#)
- Instrumentation Engineering (IN): [IN_GATE2027_Syllabus.pdf](#)
- General Aptitude, common to every paper (GA): [GA_GATE2027_Syllabus.pdf](#)

Where the CS syllabus meets this guide

CS syllabus section	Chapters
Engineering mathematics: discrete mathematics, linear algebra, probability	11 (codes and the Kraft inequality), 13 (lattices and fixed points), 15 (vectors, unitary matrices, eigenvalues)
Digital logic: Boolean algebra, circuits, number representation and computer arithmetic	9, 14
Computer organisation and architecture: instructions and addressing, data path and control unit, pipelining	13 (microprogrammed control), 14
Programming and data structures	the examples throughout, in Python and C
Algorithms: graph algorithms and complexity, P and NP	13 (graph colouring), 15 (Max-Cut)
Theory of computation: regular and context-free languages, automata, Turing machines	12 (rewrite systems), 13 (automata and grammars)
Compiler design: lexing, parsing, syntax-directed translation, intermediate code, local optimisation, dataflow analysis	13, directly

CS syllabus section	Chapters
Operating systems	8 (namespaces and control groups), 14 (the boot chain)
Computer networks	9 (network byte order); not otherwise covered
Databases	not covered

For Robotics and Automation, read the RA syllabus beside the estate repositories that meet it in practice: [zistgah/pench](#), the embodied cyler; [zistgah/transeg](#), the local digital twin; and [zistgah/jugaad28](#) and [zistgah/zamin](#) on the hardware side.

Courses and the reference lab

Courses at GATE level for both CS and RA, aligned with the reference lab and its dockerised launch scripts, are **planned** as the next step after this guide. They will be announced in the [zistgah](#) organisation.

27. Syllabus links, and our syllabus

In plain words. Links from your school or exam syllabus to the chapters that teach the same ideas, worked out on your own device, and a draft syllabus of our own.

A student's syllabus and this guide cover many of the same ideas under different names. This chapter links them without copying a word of any syllabus: the guide stores only the official address of each syllabus and its own list of keywords, and the matching happens on the reader's own device.

How the link works. Choose a board or exam; the page opens its official syllabus, from the publisher's own site. If the browser may read it directly, the page reads it; if the publisher does not allow that, which most do not, save the syllabus from the official page and drop the file on this page. Either way, the text is read in your browser, matched against the guide's own keywords, and every match becomes a link into the chapter that teaches it. Nothing is uploaded, stored or sent anywhere, and the syllabus remains the property of its publisher.

The boards and exams linked

Board or exam	Body	Levels	Region	Official page
GATE 2027, Computer Science and Information Technology	IIT Madras	Graduate	India	gate2027.iitm.ac.in
GATE 2027, Robotics and Automation	IIT Madras	Graduate	India	gate2027.iitm.ac.in
GATE 2027, Instrumentation Engineering	IIT Madras	Graduate	India	gate2027.iitm.ac.in
GATE 2027, General Aptitude	IIT Madras	Graduate	India	gate2027.iitm.ac.in
GATE 2027, every paper	IIT Madras	Graduate	India	gate2027.iitm.ac.in
CBSE, classes 9 to 12	Central Board of Secondary Education	Secondary, senior secondary	India	cbseacademic.nic.in
NCERT textbooks	National Council of Educational Research and Training	Classes 1 to 12	India	ncert.nic.in
ICSE, class 10	Council for the Indian School Certificate Examinations	Secondary	India	cisce.org

Board or exam	Body	Levels	Region	Official page
ISC, class 12	Council for the Indian School Certificate Examinations	Senior secondary	India	cisce.org
NIOS	National Institute of Open Schooling	Secondary, senior secondary	India	www.nios.ac.in
Cambridge IGCSE	Cambridge International Education	Secondary	International	www.cambridgeinternational.org
Cambridge International AS and A Level	Cambridge International Education	Senior secondary	International	www.cambridgeinternational.org
IB Diploma Programme	International Baccalaureate	Senior secondary	International	www.ibo.org
IB Middle Years Programme	International Baccalaureate	Middle, secondary	International	www.ibo.org
Advanced Placement courses	College Board	Senior secondary	United States	apcentral.collegeboard.org
Common Core State Standards	National Governors Association and CCSSO	Kindergarten to grade 12	United States	www.thecorestandards.org
AQA GCSE and A level	AQA	Secondary, senior secondary	United Kingdom	www.aqa.org.uk
Pearson Edexcel qualifications	Pearson	Secondary, senior secondary	United Kingdom	qualifications.pearson.com
OCR qualifications	OCR	Secondary, senior secondary	United Kingdom	www.ocr.org.uk
Australian Curriculum, version 9	ACARA	Foundation to year 10	Australia	v9.australiancurriculum.edu.au
Singapore national examinations	SEAB	Secondary, pre-university	Singapore	www.seab.gov.sg
Telangana Board of Secondary Education	Government of Telangana	Secondary	India, state board	bse.telangana.gov.in
Andhra Pradesh Board of Secondary Education	Government of Andhra Pradesh	Secondary	India, state board	bse.ap.gov.in
Maharashtra State Board	Government of Maharashtra	Secondary, higher secondary	India, state board	mahahsscboard.in

Board or exam	Body	Levels	Region	Official page
Karnataka School Examination and Assessment Board	Government of Karnataka	Secondary	India, state board	kseab.karnataka.gov.in
Tamil Nadu Directorate of Government Examinations	Government of Tamil Nadu	Secondary, higher secondary	India, state board	dge.tn.gov.in
Kerala SCERT	Government of Kerala	School	India, state board	scert.kerala.gov.in
West Bengal Board of Secondary Education	Government of West Bengal	Secondary	India, state board	wbbse.wb.gov.in
Uttar Pradesh Madhyamik Shiksha Parishad	Government of Uttar Pradesh	Secondary, intermediate	India, state board	upmsp.edu.in
Board of Secondary Education, Rajasthan	Government of Rajasthan	Secondary, senior secondary	India, state board	rajeduboard.rajasthan.gov.in
Gujarat Secondary and Higher Secondary Education Board	Government of Gujarat	Secondary, higher secondary	India, state board	www.gseb.org
Punjab School Education Board	Government of Punjab	Secondary, senior secondary	India, state board	www.pseb.ac.in

The GATE papers' addresses were retrieved for this guide on 29 September 2026; the other addresses are the publishers' own sites, where the current syllabus is published. Each carries its review mark in the register:

Our syllabus, a draft for review

A standard syllabus built from this guide, level by level, with its own outcomes, chapters and laboratory work. It is a draft for the author's review, and each module carries its review mark in the register at the end of the guide.

Foundation, classes 6 to 8

Module	Title	Hours	Outcomes	In this guide	Laboratory
F1	Numbers inside the machine	6	Count in binary and convert to decimal; Explain why a computer needs only two symbols	Chapter 17	

Module	Title	Hours	Outcomes	In this guide	Laboratory
F2	Letters, scripts and your language	6	Show that every letter is stored as a number; Write a word in two scripts and say what stays the same	Chapter 18, Chapter 19	
F3	The sky and the calendar	4	Read sunrise, moonrise and the tithi for a day; Explain why calendars differ	Chapter 10	chakra, jyotish
F4	Making with a cycler, safely	4	Make a short story or picture with a cycler; Say what you checked before keeping the result	Chapter 24	genie

Secondary, classes 9 and 10

Module	Title	Hours	Outcomes	In this guide	Laboratory
S1	Binary, logic and circuits	10	Build truth tables for basic gates; Relate a gate to transistors	Chapter 17, Chapter 22	
S2	Encoding and reversible transliteration	8	Explain UTF-8; Run a Romenagri round trip and judge whether it is exact	Chapter 18, Chapter 19	hindawi
S3	Programming in your own language	10	Compile and run a Hindi program; Find the same names in the debugger	Chapter 21	hindawi, urdu
S4	Using AI and checking it	6	Use any AI on a task and record each step; Tell an AI's claim from a verified result	Chapter 24, Chapter 8	alam

Senior secondary, classes 11 and 12

Module	Title	Hours	Outcomes	In this guide	Laboratory
H1	Grammar as a formal system	10	Write a regular expression and a small grammar; Read a Pāṇinian rule as a rewrite rule	Chapter 20, Chapter 21	tajziya
H2	From source code to machine	12	Trace a program from source through compiler, assembler and linker; Read symbols in an ELF file	Chapter 21, Chapter 22	hindawi
H3	Open source, provenance and contributing	6	Make a commit, open a pull request and pass a gate; Explain a hash, a manifest and a DOI	Chapter 25	rahnuma
H4	A project seeded from FAKIR	15	Choose a domain, a language and a layer and state a gate; Deliver the work with its evidence	Chapter 29	fakir

Undergraduate and GATE

Module	Title	Hours	Outcomes	In this guide	Laboratory
U1	Theory of computation and compilers	30	Automata, grammars and parsing; Build and debug a compiler pipeline	Chapter 20, Chapter 21, Chapter 26	hindawi , panini_by_claude
U2	Digital logic and architecture	30	From gates to a processor; Ternary logic and its trade-offs	Chapter 22, Chapter 26	pratik , zamin
U3	Quantum and physical computing	20	Qubits, gates and QAOA; Oscillator networks and Ising problems	Chapter 23	paniniq

Postgraduate and research

Module	Title	Hours	Outcomes	In this guide	Laboratory
P1	Research on the open questions	60	Choose an open question from Appendix C or D; Publish a measured, reproducible result	Appendix C, Appendix D, Appendix F	paniniq , tajziya

Professional

Module	Title	Hours	Outcomes	In this guide	Laboratory
Q1	Adopting a component	12	Run a component's gate; Integrate it with its contract and provenance intact	Chapter 15, Chapter 25, Appendix E	dhancha , humanesque

© 1993–2026 Abhishek Choudhary. All rights reserved. AyeAI. Text CC BY-SA 4.0; scripts GPL-3.0-or-later. Version 2.0.0, 3 October 2026.